



Structural Verilog Recap

- Structural Verilog is simply instantiation and wiring of cells/blocks whose functionality is known.
 - The cells/blocks might be built in primitives (AND, OR, NOT, ...) or might be more complex cells that you defined using dataflow or behavioral Verilog.
- Built in primitives: **and, or, not, nand, nor, xor, xnor, bufif1, notif1, bufif0, notif0**
 - Output port appears before input ports
 - Optionally specify: instance name, delay, signal strength

```
and N25 (Z, A, B);           // name specified
and #10 (Z, A, B), (X, C, D); // delay specified, 2 gates
and #10 N30 (Z, A, B);       // delay and name specified
and #10 (pull0,strong1) N35 (Z, A, B); // delay, strength, name specified
```



Testbench Recap

- At a minimum, a testbench has to do 3 things:

1. Instantiate the DUT
2. Hook up the DUT
3. Apply stimulus to DUT inputs

```
module basic_test_bench();                                // a testbench is self-contained (has no inputs/outputs)

  logic stim1, stim2;                                     // stimulus to DUT declared as type reg or logic

  ...
  // Instantiate DUT = Device Under Test //
  DUT_module iNAME(.clk(clk), .rst_n(rst_n), .in1(stim1), .in2(stim2), .out1());

  initial begin                                           // stimulus typically provided in an initial block
    stim1 = 0;
    stim2 = 1;
    #5;                                                    // wait 5 time units then change stimulus
    stim1 = 1;
    @(posedge clk); $stop();                               // wait until positive edge of clk then stop the simulation
  end

  always
    #5 clk = ~clk;    // toggle clock every 5 time units

endmodule
```

module name of DUT
Does not have to match file name

Instance name can be anything, but is required

Input of DUT named in1 is hooked to a signal called stim1 at this level of hierarchy

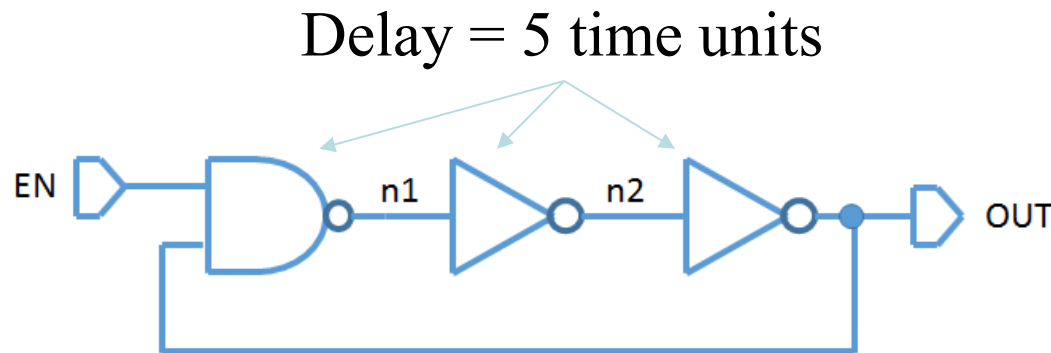
Outputs of DUT can be hooked to nothing, however, more likely you would hook it to a signal so you can monitor it



Exercise 02: Structural Verilog

Part 1: Ring Oscillator

- Implement the following DUT using structural Verilog (ring_osc.sv)



- Assuming EN has been low for a while, what does the circuit do when you raise EN?
- Create a testbench to instantiate and test it (ring_osc_tb.sv)
 - EN should be low for 15 time units and then raised
- Capture the waveforms (waves.jpg) proving it works

→ Submit ring_osc.sv, ring_osc_tb.sv, waves.jpg



- What is the functionality of this circuit?
(Hint: it is a common logic element represented at the gate level)
- Code this circuit structurally in Verilog. Call the file MSFF.sv
 - Which primitive corresponds to an inverting tri-state with an active high enable?
 - What is the order of ports for tri-states?
 - How does one specify a primitive as being weak?
- Create a testbench (MSFF_tb.sv), instantiate and simulate this circuit.
- For HW1 you will be asked to add an asynch active high reset signal. Think about how you would do that.



Exercise 02: Structural Verilog

Part 2: HW1 – last question

- Note:
 - Tri-states might need a 1 time unit delay modeled for proper simulation
 - One thing type **logic** cannot do is resolve multiple drivers. Nodes **md** & **sd** have the tri-state and weak feedback inverter driving. This has to be resolved in simulation to see “who wins”. Only type **wire** in verilog can handle resolution of multiple drivers, so nodes **md** & **sd** have to be declared as type **wire** instead of **logic**.

→ Submit MSFF.sv, MSFF_tb.sv